

```
# Check whether it's Tesla T4
!nvidia-smi

# Tweak model to Google T4: Edit -> Notebook Settings -> T4 GPU
```

Wed Jun 24 12:26:09 2026

```
+-----+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07      CUDA Version: 13.0     |
+-----+-----+-----+-----+-----+-----+
| GPU  Name            Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap|         Memory-Usage | GPU-Util  Compute M. |
|                                           | MIG M.           |
+-----+-----+-----+-----+-----+-----+
|   0   Tesla T4            Off      | 00000000:00:04:0 Off |                    0 |
| N/A   43C    P8             10W /  70W |  0MiB / 15360MiB |      0%      Default |
+-----+-----+-----+-----+-----+-----+
|                                           | N/A              |
```

```
+-----+
| Processes:                                |
| GPU   GI    CI          PID    Type   Process name                      GPU Memory |
| ID   ID     ID              |          |                     Usage        |
+-----+-----+-----+-----+-----+-----+
| No running processes found |
```

```
# pip install necessary items
!pip install ultralytics roboflow -q
import ultralytics, os, roboflow
from ultralytics import YOLO
from roboflow import Roboflow
```

```
_____ 41.3/41.3 kB 2.4 MB/s eta 0:00:00
_____ 1.3/1.3 MB 32.1 MB/s eta 0:00:00
_____ 250.0/250.0 kB 23.8 MB/s eta 0:00:00
_____ 66.8/66.8 kB 6.7 MB/s eta 0:00:00
_____ 49.9/49.9 MB 20.2 MB/s eta 0:00:00
_____ 1.5/1.5 MB 87.1 MB/s eta 0:00:00
_____ 5.5/5.5 MB 125.2 MB/s eta 0:00:00
```

Creating new Ultralytics Settings v0.0.6 file 

View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'

Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see <https://docs.ultralytics.com>.

```
# Introduce Roboflow api
```

```
rf = Roboflow(api_key="Your_API_KEY")
project = rf.workspace("fyp-ormnr").project("supermarket-empty-shelf-detector")
version = project.version(4)
dataset = version.download("yolov11")
```

loading Roboflow workspace...

loading Roboflow project...

Downloading Dataset Version Zip in Supermarket-Empty-Shelf-Detector--4 to yolov11:: 100%|██████████| 45056/45056 [00:00<

Extracting Dataset Version Zip to Supermarket-Empty-Shelf-Detector--4 in yolov11:: 100%|██████████| 1006/1006 [00:00<

```
# # tweak the Roboflow path with both val and valid
# path = 'content/Supermarket-Empty_shelf-Detector--4'
# dataset.location = os.path.join(path, 'yaml') # os.path.join('user', folder, filename)
```

```
# Download YOLO11n and run train the data
```

```
model = YOLO("yolo11n.pt")
```

```
results = model.train(data=os.path.join(dataset.location, "data.yaml"), epochs=100, patience = 50 )
```

Downloading <https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt> to 'yolo11n.pt': 100% _____

Ultralytics 8.4.75 🚀 Python-3.12.13 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)

engine/trainer: agnostic_nms=False, amp=True, angle=1.0, augment=False, auto_augment=randaugument, batch=16, bgr=0.0,

Downloading <https://ultralytics.com/assets/Arial.ttf> to '/root/.config/Ultralytics/Arial.ttf': 100% _____ 75!

Overriding model.yaml nc=80 with nc=1

	from	n	params	module	arguments
0	-1	1	464	ultralytics.nn.modules.conv.Conv	[3, 16, 3, 2]
1	-1	1	4672	ultralytics.nn.modules.conv.Conv	[16, 32, 3, 2]
2	-1	1	6640	ultralytics.nn.modules.block.C3k2	[32, 64, 1, False, 0.25]
3	-1	1	36992	ultralytics.nn.modules.conv.Conv	[64, 64, 3, 2]

```

4          -1 1      26080 ultralytics.nn.modules.block.C3k2      [64, 128, 1, False, 0.25]
5          -1 1     147712 ultralytics.nn.modules.conv.Conv      [128, 128, 3, 2]
6          -1 1      87040 ultralytics.nn.modules.block.C3k2      [128, 128, 1, True]
7          -1 1     295424 ultralytics.nn.modules.conv.Conv      [128, 256, 3, 2]
8          -1 1     346112 ultralytics.nn.modules.block.C3k2      [256, 256, 1, True]
9          -1 1     164608 ultralytics.nn.modules.block.SPPF      [256, 256, 5]
10         -1 1     249728 ultralytics.nn.modules.block.C2PSA      [256, 256, 1]
11         -1 1         0 torch.nn.modules.upsampling.Upsample      [None, 2, 'nearest']
12        [-1, 6] 1         0 ultralytics.nn.modules.conv.Concat      [1]
13         -1 1     111296 ultralytics.nn.modules.block.C3k2      [384, 128, 1, False]
14         -1 1         0 torch.nn.modules.upsampling.Upsample      [None, 2, 'nearest']
15        [-1, 4] 1         0 ultralytics.nn.modules.conv.Concat      [1]
16         -1 1      32096 ultralytics.nn.modules.block.C3k2      [256, 64, 1, False]
17         -1 1      36992 ultralytics.nn.modules.conv.Conv      [64, 64, 3, 2]
18        [-1, 13] 1         0 ultralytics.nn.modules.conv.Concat      [1]
19         -1 1      86720 ultralytics.nn.modules.block.C3k2      [192, 128, 1, False]
20         -1 1     147712 ultralytics.nn.modules.conv.Conv      [128, 128, 3, 2]
21        [-1, 10] 1         0 ultralytics.nn.modules.conv.Concat      [1]
22         -1 1     378880 ultralytics.nn.modules.block.C3k2      [384, 256, 1, True]
23        [16, 19, 22] 1     430867 ultralytics.nn.modules.head.Detect      [1, 16, None, [64, 128, 256]]
YOLO11n summary: 182 layers, 2,590,035 parameters, 2,590,019 gradients, 6.4 GFLOPs

```

Transferred 448/499 items from pretrained weights

Freezing layer 'model.23.dfl.conv.weight'

AMP: running Automatic Mixed Precision (AMP) checks...

Downloading <https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo26n.pt> to 'yolo26n.pt': 100% —

AMP: checks passed ✓

train: Fast image access ✓ (ping: 0.0±0.0 ms, read: 2001.0±1348.8 MB/s, size: 89.4 KB)

train: Scanning /content/Supermarket-Empty-Shelf-Detector--4/train/labels... 350 images, 0 backgrounds, 0 corrupt: :

train: New cache created: /content/Supermarket-Empty-Shelf-Detector--4/train/labels.cache

augmentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01, blur_limit=(3, 7)), ToGray(p=0.01, method='weight')

val: Fast image access ✓ (ping: 0.0±0.0 ms, read: 1075.2±639.3 MB/s, size: 86.0 KB)

val: Scanning /content/Supermarket-Empty-Shelf-Detector--4/valid/labels... 97 images, 0 backgrounds, 0 corrupt: 100%

val: New cache created: /content/Supermarket-Empty-Shelf-Detector--4/valid/labels.cache

optimizer: 'optimizer=auto' found, ignoring 'lr0=0.01' and 'momentum=0.937' and determining best 'optimizer', 'lr0'

optimizer: AdamW(lr=0.002, momentum=0.9) with parameter groups 81 weight(decay=0.0), 88 weight(decay=0.0005), 87 bias

Plotting labels to /content/runs/detect/train/labels.jpg...

Image sizes 640 train, 640 val

Using 2 dataloader workers

Logging results to /content/runs/detect/train

Starting training for 100 epochs...

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
1/100	2.32G	2.454	3.594	1.755	137	640: 100% — 22/22 1.2s/it 27.5s
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% — 4/4 1.8s
	all	97	600	0.0148	0.718	0.059 0.021

Check mAP 50 and mAP50-95

metrics = model.val(data = os.path.join(dataset.location, "data.yaml"))

print(f"mAP50: {metrics.box.map50:.3f}")

print(f"mAP50-95: {metrics.box.map:.3f}")

Ultralytics 8.4.75 🚀 Python-3.12.13 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)

YOLO11n summary (fused): 101 layers, 2,582,347 parameters, 0 gradients, 6.3 GFLOPs

val: Fast image access ✓ (ping: 0.0±0.0 ms, read: 2154.7±625.3 MB/s, size: 90.0 KB)

val: Scanning /content/Supermarket-Empty-Shelf-Detector--4/valid/labels.cache... 97 images, 0 backgrounds, 0 corrupt:

Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100% — 7/7 1.8s
all	97	600	0.88	0.855	0.91	0.656

Speed: 7.8ms preprocess, 8.7ms inference, 0.0ms loss, 3.3ms postprocess per image

Results saved to /content/runs/detect/val

mAP50: 0.910

mAP50-95: 0.656

!ls /content/Supermarket-Empty-Shelf-Detector--4

!cat /content/Supermarket-Empty-Shelf-Detector--4/data.yaml

train: ../train/images

val: ../valid/images

test: ../test/images

nc: 1

names: ['100- 0-0-S']

roboflow:

workspace: fyp-ormnr

project: supermarket-empty-shelf-detector

version: 4

license: CC BY 4.0

url: <https://universe.roboflow.com/fyp-ormnr/supermarket-empty-shelf-detector/dataset/4>

```
# Display the valid images with boxes
import glob
import cv2
import matplotlib.pyplot as plt
from ultralytics import YOLO

def validate_images(path, img_count = -1):
    # 1. Grab all images from your validation folder
    val_images = glob.glob(path)

    # 2. Run inference on a few validation images
    results = model.predict(source=val_images[:img_count], conf=0.25)

    # 3. Display the images with bounding boxes
    for r in results:
        # Plot returns a BGR numpy array with boxes drawn
        im_bgr = r.plot()

        # Convert BGR to RGB for correct colors in Matplotlib/Notebooks
        im_rgb = cv2.cvtColor(im_bgr, cv2.COLOR_BGR2RGB)

        # Show using matplotlib
        plt.figure(figsize=(10, 10))
        plt.imshow(im_rgb)
        plt.axis("off")
        plt.show()

validate_images("/content/Supermarket-Empty-Shelf-Detector--4/valid/images/*.jpg", 3)
```


image 1/3 /content/Supermarket-Empty-Shelf-Detector--4/valid/images/test2020_20_jpg.rf.26bca26f40a2f0cd572872ad6081e
 image 2/3 /content/Supermarket-Empty-Shelf-Detector--4/valid/images/test2020_1071_jpg.rf.ce590da92ef8f7db7a39114686:
 image 3/3 /content/Supermarket-Empty-Shelf-Detector--4/valid/images/test2020_121_jpg.rf.5344e411b28cec24d9e438f667b
 Speed: 1.8ms preprocess, 8.0ms inference, 1.2ms postprocess per image at shape (1, 3, 640, 640)



Check empty area/ overall area rate

```
def empty_area_rate(path, img_count = -1):
    # 1. Grab all images from your validation folder
    val_images = glob.glob(path)

    # 2. Run inference on a few validation images
    results = model.predict(source=val_images[:img_count], conf=0.25)

    # 3. Display the images with bounding boxes
    img = 1
    for r in results:
        h_img, w_img = r.orig_shape
        ttl_boxes_area = 0
        for _, _, w_box, h_box in r.bboxes.xywh:
            ttl_boxes_area += h_box*w_box
        rate = round(float(100* ttl_boxes_area / (h_img*w_img)), 2)
        print(f"Image {img} has {rate}% of total area of the image is empty.")
        img += 1
    empty_area_rate("/content/Supermarket-Empty-Shelf-Detector--4/valid/images/*.jpg", 3)
```

image 1/3 /content/Supermarket-Empty-Shelf-Detector--4/valid/images/test2020_20_jpg.rf.26bca26f40a2f0cd572872ad6081e6
 image 2/3 /content/Supermarket-Empty-Shelf-Detector--4/valid/images/test2020_1071_jpg.rf.ce590da92ef8f7db7a3911468635

```
image 3/3 /content/Supermarket-Empty-Shelf-Detector--4/valid/images/test2020_121_jpg.rf.5344e411b28cec24d9e438f667bfb
Speed: 2.1ms preprocess, 12.7ms inference, 1.6ms postprocess per image at shape (1, 3, 640, 640)
Image 1 has 2.57% of total area of the image is empty.
Image 2 has 7.35% of total area of the image is empty.
Image 3 has 3.51% of total area of the image is empty.
```

```
# Time to test my own images!

# Create a folder and upload my own images to the folder
from google.colab import files
import shutil

# Make a new folder for my images
path = '/content/Supermarket-Empty-Shelf-Detector--4/my_images'
os.makedirs(path, exist_ok = True)

# This opens the upload prompt
uploaded = files.upload()

# This automatically moves your uploaded images into the folder you made
for filename in uploaded.keys():
    shutil.move(filename, path)
```

No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving IMG_9458.HEIC to IMG_9458.HEIC
Saving IMG_9459.HEIC to IMG_9459.HEIC
Saving IMG_9460.HEIC to IMG_9460.HEIC
Saving IMG_9461.HEIC to IMG_9461.HEIC
Saving IMG_9462.HEIC to IMG_9462.HEIC
Saving IMG_9463.HEIC to IMG_9463.HEIC
Saving IMG_9464.HEIC to IMG_9464.HEIC
Saving IMG_9465.HEIC to IMG_9465.HEIC
```

```
# !ls /content/Supermarket-Empty-Shelf-Detector--4/my_images
```